

Barrier vs. Simplex for LP Relaxations in Clp

Comparing Native Cholesky, AMD Cholesky, Primal Simplex,
and Dual Simplex on MIPLIB 2017 Instances

Experimental study — CBC/CLP development workspace

March 2026

Contents

1	Motivation	1
2	Experimental Setup	1
2.1	Solver	1
2.2	Solver Configurations	1
2.3	Time Limits	2
2.4	Instances	2
2.5	Objective Validation	2
2.6	Measurement	2
3	Results	3
3.1	Solve Time	3
3.2	Wall-clock Time	3
3.3	Peak Memory	3
3.4	Iteration Counts	3
4	Analysis	4
4.1	Primal Simplex Dominates	4
4.2	Barrier AMD Is Uniformly Worse Than Native Barrier	4
4.3	The <code>seymour</code> Pathology	5
4.4	The <code>graph40-80-1rand</code> Case	5
4.5	Network Instances: <code>momentum1</code>	6
4.6	Memory Overhead	6
5	Conclusions and Recommendations	6
5.1	Main Conclusions	6
5.2	Recommendations for <code>guess()</code> Extension	7
5.3	AMD Cholesky	7

1 Motivation

The `-guess` option in Clp (documented separately in `doc/guess_option.pdf`) automatically selects an LP algorithm based on instance features. In its current form it always chooses either primal or dual simplex; the barrier (interior-point) branch is present in the code but is dead. A natural question is: *is that correct?* When, if ever, does barrier outperform simplex on the LP relaxations that arise inside a MIP solve?

A secondary question is whether the SuiteSparse / CHOLMOD AMD reordering (available in this build via `-chol uni`) improves the barrier solver relative to Clp’s native Cholesky, which is relevant both for speed and for peak memory consumption.

This report documents a systematic experiment that benchmarks four solver configurations on eleven MIPLIB 2017 instances that are cited in the `guess_option` document as structural candidates for the barrier method.

2 Experimental Setup

2.1 Solver

All runs use the `clp` binary from the local development build (`$HOME/prog/cbc/bin/clp`, version *devel*, build date 2026-03-24), compiled with:

- GCC 15 (`-O3 -march=native`);
- SuiteSparse / AMD (`libamd`, `libcholmod`) from the system package;
- OpenBLAS for dense linear algebra.

2.2 Solver Configurations

Four Clp configurations are tested, differing only in LP algorithm and Cholesky ordering:

Table 1: Solver configurations.

Label	Clp command-line flags	Description
<code>barrier_default</code>	<code>-seconds 28800 -barrier</code>	Barrier, native Cholesky
<code>barrier_chol_amd</code>	<code>-chol uni -seconds 28800 -barrier</code>	Barrier, CHOLMOD + AMD ordering
<code>primal_simplex</code>	<code>-seconds 28800 -primals</code>	Primal simplex (default crash)
<code>dual_simplex</code>	<code>-seconds 28800 -duals</code>	Dual simplex

Important: In Clp, `-barrier` and `-primals/-duals` are *action* parameters that trigger the solve immediately. All configuration parameters (including `-chol` and `-seconds`) must therefore precede the action flag.

2.3 Time Limits

- **Soft limit:** 28 800 s (8 h), passed to Clp via `-seconds`.
- **Hard limit:** 32 400 s (9 h), enforced externally via `SIGTERM/SIGKILL`.

2.4 Instances

The eleven instances in Table 2 were selected from MIPLIB 2017 based on structural properties suggested as barrier-friendly in the `guess_option` document: high density, network/flow structure, high degeneracy, and high equality proportion. All instances are taken from `/home/haroldo/inst/super` in `.mps.gz` format.

2.5 Objective Validation

After each solve that reached optimality, the reported objective value was compared against a Gurobi LP-relaxation reference (`lp_relaxation_results_gurobi.tsv`) using a relative tolerance of 10^{-4} . All optimal solutions in this experiment matched within 10^{-10} , confirming correctness across all four configurations.

Table 2: Benchmark instances.

Instance	Category	Structural rationale
neos-5075914-elvire	Dense / large-scale	Large, dense constraint matrix
neos-3083819-nubu	Dense / large-scale	High density, many nz per row
supportcase18	Dense / large-scale	Large-scale, dense blocks
graph40-80-1rand	Dense / large-scale	Dense graph-based structure
mc11	Network / flow	Multicommodity flow
momentum1	Network / flow	Large network structure
neos-4954672-berkel	Network / flow	Flow-like, many equalities
supportcase6	Network / flow	Large flow/network relaxation
neos-3627168-kasai	Highly degenerate	Degenerate LP, simplex stalls
neos-4338804-snowy	Highly degenerate	Degenerate, simplex struggles
seymour	Highly degenerate	Classic difficult degenerate LP

2.6 Measurement

- **Wall-clock time:** measured externally using millisecond timestamps around the solver call.
- **Solve time:** Clp-reported time on the `Optimal objective ... time` output line, excluding presolve.
- **Peak memory:** Maximum resident set size from `/usr/bin/time -v` wrapping the Clp process.

All 44 jobs (11 instances $\times$ 4 configurations) ran concurrently on a 128-core machine; each Clp process is single-threaded, so at most 44 cores were active simultaneously.

3 Results

3.1 Solve Time

Table 3 reports Clp-reported solve time (excluding presolve). The fastest configuration for each instance is highlighted in **green**; time-limit hits are in **grey**.

The geometric mean ratio measures each configuration’s solve time relative to the fastest on that instance, computed only over jointly solved instances. A value of 1.000 would mean always fastest.

3.2 Wall-clock Time

Wall-clock times (Table 4) follow the same pattern but include startup, I/O, and presolve overhead. The rankings are essentially identical to solve time.

3.3 Peak Memory

Table 5 reports the peak resident set size. The most striking differences appear on the three hardest instances (Table 6).

3.4 Iteration Counts

Table 7 reports the number of barrier (IPM) or simplex iterations. Barrier’s theoretical advantage is its dimension-independent iteration count.

Table 3: Clp-reported solve time (seconds, excluding presolve). TL = time limit reached. Best per row highlighted.

Instance	Barrier native	Barrier+AMD	Primal	Dual	Best
neos-5075914-elvire	0.102	0.142	0.052	0.072	Primal
neos-3083819-nubu	0.092	0.472	0.042	0.032	Dual
supportcase18	0.152	0.192	0.042	0.152	Primal
graph40-80-1rand	25 452	TL	14 148	TL	Primal
mc11	0.012	0.022	0.012	0.012	Barrier native
momentum1	TL	TL	4.842	0.602	Dual
neos-4954672-berkel	0.022	0.182	0.012	0.012	Primal
supportcase6	6.742	7.582	1.302	3.642	Primal
neos-3627168-kasai	0.032	0.062	0.042	0.032	Barrier native
neos-4338804-snowy	0.612	0.432	0.012	0.012	Primal
seymour	47.8	796	0.462	0.872	Primal
Wins	2	0	7	2	
Timed out	1	2	0	1	
GeoMean ratio	4.231	11.524	1.270	1.388	

Table 4: Wall-clock time (seconds). TL = time limit reached.

Instance	Barrier native	Barrier+AMD	Primal	Dual	Best
neos-5075914-elvire	0.164	0.185	0.089	0.112	Primal
neos-3083819-nubu	0.173	0.512	0.093	0.076	Dual
supportcase18	0.231	0.247	0.099	0.178	Primal
graph40-80-1rand	25 529	TL	14 219	TL	Primal
mc11	0.047	0.051	0.045	0.045	Primal
momentum1	TL	TL	5.063	0.766	Dual
neos-4954672-berkel	0.053	0.197	0.036	0.053	Primal
supportcase6	7.176	8.023	1.743	4.110	Primal
neos-3627168-kasai	0.081	0.099	0.116	0.064	Dual
neos-4338804-snowy	0.627	0.461	0.072	0.036	Dual
seymour	47.842	796.738	0.506	0.947	Primal

The barrier method achieves remarkably few iterations on most instances (6–53 for the easy ones, 11 for `graph40-80-1rand`). However, each iteration involves a Cholesky factorisation that can be orders of magnitude more expensive per iteration than a simplex pivot, explaining why low iteration counts do not translate into fast wall-clock times.

4 Analysis

4.1 Primal Simplex Dominates

Primal simplex is the fastest method on 7 of 11 instances and is the only configuration to solve all 11 within the time limit. Its geometric mean ratio to the best (1.27) is far lower than barrier native (4.23) or barrier+AMD (11.52). This is consistent with the design of the existing `guess()` heuristic, which predominantly recommends primal simplex.

Table 5: Peak memory usage (MB). $\uparrow$ highest in row; $\downarrow$ lowest in row.

Instance	Barrier native	Barrier+AMD	Primal	Dual
neos-5075914-elvire	16	19 $\uparrow$	17	17
neos-3083819-nubu	20	19	21 $\uparrow$	21 $\uparrow$
supportcase18	18	23 $\uparrow$	19	18
graph40-80-1rand	8 741	12 533 $\uparrow$	862 $\downarrow$	886
mc11	17 $\uparrow$	16	17 $\uparrow$	14 $\downarrow$
momentum1	786	967 $\uparrow$	46 $\downarrow$	50
neos-4954672-berkel	16	17 $\uparrow$	15 $\downarrow$	17 $\uparrow$
supportcase6	82	81 $\downarrow$	82	84 $\uparrow$
neos-3627168-kasai	16	17 $\uparrow$	17 $\uparrow$	14 $\downarrow$
neos-4338804-snowy	21	22 $\uparrow$	15 $\downarrow$	16
seymour	65	97 $\uparrow$	17 $\downarrow$	20

Table 6: Memory overhead of barrier vs. primal simplex on hard instances.

Instance	Barrier native	Barrier+AMD	Primal	AMD/Primal
graph40-80-1rand	8 741 MB	12 533 MB	862 MB	$\times 14.5$
momentum1	786 MB	967 MB	46 MB	$\times 21.0$
seymour	65 MB	97 MB	17 MB	$\times 5.7$

4.2 Barrier AMD Is Uniformly Worse Than Native Barrier

On all 9 instances where both barrier variants solved to optimality, native Cholesky was faster than AMD Cholesky (Table 8). The ratio **AMD/native** ranges from $1.1\times$ to $16.7\times$, with a median around $1.9\times$. Only on **neos-4338804-snowy** (a trivially fast instance where both solve in under 1s) does AMD show any advantage ($0.71\times$).

The AMD ordering is designed to minimise fill-in in the Cholesky factor of the normal-equations matrix $AD^{-2}A^\top$. The results suggest that at these problem scales the reordering cost exceeds the factorisation savings, or that the sparsity pattern of the normal equations for these instances is not amenable to AMD reordering. The consistently higher memory usage of the AMD variant (despite its fill-reduction goal) further indicates the reordering is not producing sparser factors in practice—possibly because the normal-equations matrix is already dense due to dense columns or degenerate structure.

4.3 The seymour Pathology

seymour is the starkest result. It is a classic highly-degenerate covering LP. Primal simplex solves it in 0.46 s (2 764 pivots); native barrier takes 47.8 s (49 iterations, $100\times$ slower); AMD barrier takes 796 s (229 iterations, $1\,730\times$ slower). The AMD variant performs *four times more barrier iterations* than native and takes $16\times$ longer per iteration, suggesting the AMD ordering creates a pathologically ill-conditioned factorisation for this problem’s normal-equations matrix.

4.4 The graph40-80-1rand Case

This is the instance where barrier comes closest to being competitive. It is a dense graph colouring LP with -512 as the LP relaxation optimum. Primal simplex requires 614 030 pivots and 3.94 h; native barrier solves it with only 11 iterations in 7.07 h. Barrier’s iteration count is $56\,000\times$ smaller, yet it is $1.8\times$ slower overall—each Cholesky factorisation of the dense normal-equations matrix costs enormously. Native barrier at least terminates; AMD barrier and dual simplex both

Table 7: Number of barrier or simplex iterations. TL = timed out.

Instance	Barrier native	Barrier+AMD	Primal	Dual
neos-5075914-elvire	20	19	1 074	1 376
neos-3083819-nubu	446	339	539	528
supportcase18	11	11	663	985
graph40-80-1rand	11	TL	614 030	TL
mc11	53	53	662	404
momentum1	TL	TL	7 293	3 138
neos-4954672-berkel	60	281	322	409
supportcase6	721	724	9 787	4 390
neos-3627168-kasai	150	159	1 294	1 366
neos-4338804-snowy	6	6	474	409
seymour	49	229	2 764	3 710

Table 8: Barrier native vs. Barrier+AMD: solve time and memory comparison. Ratio > 1 means AMD is slower.

Instance	Native (s)	AMD (s)	Ratio	Mem native	Mem AMD	Mem ratio
neos-5075914-elvire	0.102	0.142	1.39×	16 MB	19 MB	1.19×
neos-3083819-nubu	0.092	0.472	5.13×	20 MB	19 MB	0.95×
supportcase18	0.152	0.192	1.26×	18 MB	23 MB	1.28×
graph40-80-1rand	25 452	TL	—	8 741 MB	12 533 MB	1.43×
mc11	0.012	0.022	1.83×	17 MB	16 MB	0.94×
momentum1	TL	TL	—	786 MB	967 MB	1.23×
neos-4954672-berkel	0.022	0.182	8.27×	16 MB	17 MB	1.06×
supportcase6	6.742	7.582	1.12×	82 MB	81 MB	0.99×
neos-3627168-kasai	0.032	0.062	1.94×	16 MB	17 MB	1.06×
neos-4338804-snowy	0.612	0.432	0.71×	21 MB	22 MB	1.05×
seymour	47.8	796	16.67×	65 MB	97 MB	1.49×

hit the 8-hour soft limit without converging, and AMD barrier consumes 12.5 GB of memory (vs. 8.7 GB native, 862 MB primal).

This instance illustrates the key trade-off: barrier achieves polynomial iteration counts on dense problems but each iteration has high per-iteration cost due to Cholesky, whereas simplex iterates cheaply but may require an exponential number of pivots in degenerate cases.

4.5 Network Instances: momentum1

Both barrier variants fail to converge on **momentum1** within 8 hours, while dual simplex solves it in 0.60 s. Network flow problems typically have a totally unimodular constraint matrix; simplex exploits this structure via fast pivots with integer intermediate solutions. Barrier offers no such structural advantage and the large, sparse network normal-equations matrix appears difficult to factor efficiently despite its special structure.

4.6 Memory Overhead

The memory cost of barrier is significant:

- On large instances, barrier uses 10–21× more memory than primal simplex.

- AMD Cholesky consistently uses *more* memory than native Cholesky, the opposite of its intended purpose, suggesting the AMD reordering is not beneficial for these problem structures.
- For a MIP solver that calls the LP relaxation at every node, barrier’s memory overhead would be multiplied across the tree.

5 Conclusions and Recommendations

5.1 Main Conclusions

1. **Primal simplex is the best default.** It is fastest on 7/11 instances, solves all 11, and has the lowest geometric mean ratio (1.27). The current `guess()` behaviour of predominantly recommending primal simplex is empirically justified.
2. **Barrier (native Cholesky) is occasionally competitive.** It ties for fastest on `mc11` and `neos-3627168-kasai`, and is the only method to solve `graph40-80-1rand` within the time limit. However, these gains come with a 10–15× memory penalty.
3. **AMD Cholesky (`-chol uni`) provides no benefit over native Cholesky** on this benchmark set. It is slower on 9/9 comparable pairs and uses more memory in most cases. It should *not* be recommended for these instance types.
4. **Barrier is dangerous for degenerate LPs.** On `seymour` and `momentum1`, it either diverges slowly or fails to converge. Including barrier in an automated selection strategy requires a reliable degeneracy detector.
5. **The dead barrier branch in `guess()` is intentional.** Given the evidence, adding a barrier leaf to the decision tree would require a strong filter (e.g., dense graph structure detected via the `OsiFeatures`) to avoid catastrophic performance on degenerate or network instances.

5.2 Recommendations for `guess()` Extension

If barrier is to be added as a `guess()` leaf, the following `OsiFeature` thresholds are suggested as a filter based on this experiment:

- Non-zeros per row > 50 (dense matrix);
- Fraction of equality rows > 0.5 ;
- Fraction of network/flow constraints < 0.3 (exclude pure networks);
- Degeneracy indicator $< \text{threshold}$ (exclude highly degenerate).

Even with such a filter, the gain over primal simplex on the one qualifying instance (`graph40-80-1rand`) is marginal (barrier is 1.8× *slower* in wall time despite needing 56 000× fewer iterations), so a barrier leaf may not improve the decision tree’s overall performance.

5.3 AMD Cholesky

The `-chol uni` option should be treated as experimental for LP relaxation solving. It is not recommended as a default for MIP LP relaxations. Further investigation on purely continuous (non-MIP) large-scale dense LPs would be needed to find a regime where it adds value.

Appendix: Raw Data

Table 9: Complete results. Status: **opt** = optimal, **TL** = soft time limit (8 h), **HTL** = hard time limit (9 h). Obj. match: relative error vs. Gurobi reference.

Instance	Config	Status	Wall (s)	Solve (s)	Presolve (s)	Iters	Mem (MB)	Rel. e
neos-5075914-elvire	barrier_default	opt	0.164	0.102	0.01	20	16	1.0e-
neos-5075914-elvire	barrier_chol_amd	opt	0.185	0.142	0.01	19	19	1.0e-
neos-5075914-elvire	primal_simplex	opt	0.089	0.052	0.01	1 074	17	1.0e-
neos-5075914-elvire	dual_simplex	opt	0.112	0.072	0.01	1 376	17	1.0e-
neos-3083819-nubu	barrier_default	opt	0.173	0.092	0.02	446	20	3.1e-
neos-3083819-nubu	barrier_chol_amd	opt	0.512	0.472	0.02	339	19	3.1e-
neos-3083819-nubu	primal_simplex	opt	0.093	0.042	0.02	539	21	3.1e-
neos-3083819-nubu	dual_simplex	opt	0.076	0.032	0.02	528	21	3.1e-
supportcase18	barrier_default	opt	0.231	0.152	—	11	18	7.1e-
supportcase18	barrier_chol_amd	opt	0.247	0.192	—	11	23	7.1e-
supportcase18	primal_simplex	opt	0.099	0.042	—	663	19	7.1e-
supportcase18	dual_simplex	opt	0.178	0.152	—	985	18	7.1e-
graph40-80-1rand	barrier_default	opt	25 529	25 452	1.97	11	8 741	0.0e+
graph40-80-1rand	barrier_chol_amd	TL	28 881	—	—	—	12 533	
graph40-80-1rand	primal_simplex	opt	14 219	14 148	2.00	614 030	883	0.0e+
graph40-80-1rand	dual_simplex	TL	28 863	—	—	—	886	
mc11	barrier_default	opt	0.047	0.012	0.00	53	17	3.7e-
mc11	barrier_chol_amd	opt	0.051	0.022	0.00	53	16	3.7e-
mc11	primal_simplex	opt	0.045	0.012	0.00	662	17	3.7e-
mc11	dual_simplex	opt	0.045	0.012	0.00	404	14	3.7e-
momentum1	barrier_default	TL	29 185	—	—	—	786	
momentum1	barrier_chol_amd	TL	29 030	—	—	—	967	
momentum1	primal_simplex	opt	5.063	4.842	0.05	7 293	46	6.2e-
momentum1	dual_simplex	opt	0.766	0.602	0.05	3 138	50	6.2e-
neos-4954672-berkel	barrier_default	opt	0.053	0.022	0.00	60	16	2.6e-
neos-4954672-berkel	barrier_chol_amd	opt	0.197	0.182	0.01	281	17	2.6e-
neos-4954672-berkel	primal_simplex	opt	0.036	0.012	0.00	322	15	2.6e-
neos-4954672-berkel	dual_simplex	opt	0.053	0.012	0.00	409	17	2.6e-
supportcase6	barrier_default	opt	7.176	6.742	—	721	82	7.7e-
supportcase6	barrier_chol_amd	opt	8.023	7.582	—	724	81	7.7e-
supportcase6	primal_simplex	opt	1.743	1.302	—	9 787	82	7.7e-
supportcase6	dual_simplex	opt	4.110	3.642	0.28	4 390	84	7.7e-
neos-3627168-kasai	barrier_default	opt	0.081	0.032	0.00	150	16	4.7e-
neos-3627168-kasai	barrier_chol_amd	opt	0.099	0.062	0.00	159	17	4.7e-
neos-3627168-kasai	primal_simplex	opt	0.116	0.042	0.00	1 294	17	4.7e-
neos-3627168-kasai	dual_simplex	opt	0.064	0.032	0.00	1 366	14	4.7e-
neos-4338804-snowy	barrier_default	opt	0.627	0.612	0.00	6	21	0.0e+
neos-4338804-snowy	barrier_chol_amd	opt	0.461	0.432	0.00	6	22	0.0e+
neos-4338804-snowy	primal_simplex	opt	0.072	0.012	0.00	474	15	0.0e+
neos-4338804-snowy	dual_simplex	opt	0.036	0.012	0.00	409	16	0.0e+
seymour	barrier_default	opt	47.842	47.772	0.01	49	65	6.2e-
seymour	barrier_chol_amd	opt	796.738	796.382	0.02	229	97	6.2e-
seymour	primal_simplex	opt	0.506	0.462	0.01	2 764	17	6.2e-
seymour	dual_simplex	opt	0.947	0.872	0.01	3 710	20	6.2e-